



Name:- Soham kulkarni

Class:- Sy-A

RollNo.:- A-58

Batch:- A4

Aim:- To Data Ingestion and Cleaning: Acquire data and prepare it for analysis.

Assignment 2:

1.What is Data Ingestion and Why is it Important in Data Science? Which Python Library is Commonly Used?

Ans:- Data ingestion is the process of collecting and importing data from different sources such as CSV files, databases, APIs, or web sources into a system for further analysis. It is an important step in data science because without proper data collection, no analysis or model building can be performed. Data ingestion ensures that data is available in a structured format so that it can be processed and analyzed efficiently. One of the most commonly used Python libraries for data ingestion and cleaning is Pandas, which provides powerful functions to load, manipulate, and clean datasets easily.

2.Why is Data Cleaning Necessary Before Performing Analysis or Machine Learning?

Ans:- Exploratory Data Analysis (EDA) is an important step in data analysis that helps analysts understand the structure, patterns, and characteristics of a dataset before performing advanced analysis or building models. Using tools like Pandas, EDA allows users to examine data by calculating summary statistics, checking for missing values, identifying outliers, and understanding relationships between variables. Functions such as `describe()`, `head()`, `info()`, and visualization techniques help in discovering useful insights about the data. EDA helps detect errors or inconsistencies in the dataset and guides analysts in selecting appropriate analysis methods. Therefore, it plays a crucial role in making data-driven decisions and improving the accuracy of further data analysis or machine learning models.

3.How do you load a CSV file into a Pandas DataFrame?

Ans:- A CSV file can be loaded into a DataFrame using the `read_csv()` function provided by the Pandas library. First, the Pandas library is imported, and then the file name or file path is passed as an argument to the function. The data is then stored in a DataFrame, which allows easy manipulation and analysis of structured data. This method is widely used in data science for reading datasets from external files into Python.

4.What is the difference between head() and tail() functions in Pandas?

Ans:- In Pandas, the head() and tail() functions are used to quickly view parts of a dataset. The head() function displays the first few rows of the DataFrame, usually the top five rows by default, which helps in understanding the structure and column names. On the other hand, the tail() function displays the last few rows of the DataFrame, which is useful for checking the end of the dataset. Both functions are helpful for quickly inspecting data without displaying the entire dataset.

5.How can you identify missing values in a dataset?

Ans:- Missing values in a dataset can be identified using functions like isnull() in the Pandas library. The isnull() function checks each value in the dataset and returns a Boolean result indicating whether the value is missing or not. When combined with functions like sum(), it provides the total number of missing values in each column. Identifying missing values is an important step in data cleaning, as it helps in deciding how to handle incomplete data before performing analysis.

```
In [ ]: from google.colab import files
        uploaded = files.upload()
```

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving StudentPerfor2.csv to StudentPerfor2.csv

```
In [ ]: import pandas as pd
```

```
In [ ]: df = pd.read_csv("StudentPerfor2.csv")
        df.head()
```

```
Out[ ]:
```

	Hours_Studied	Attendance	Parental_Involvement	Access_to_Resources	Extr
0	23	84	Low	High	
1	19	64	Low	Medium	
2	24	98	Medium	Medium	
3	29	89	Low	Medium	
4	19	92	Medium	Medium	

```
In [ ]: df.isnull().sum()
```

Out[]: 0

Hours_Studied	0
Attendance	0
Parental_Involvement	0
Access_to_Resources	0
Extracurricular_Activities	0
Sleep_Hours	3
Previous_Scores	0
Motivation_Level	1
Internet_Access	0
Tutoring_Sessions	1
Family_Income	2
Teacher_Quality	80
School_Type	1
Peer_Influence	1
Physical_Activity	4
Learning_Disabilities	1
Parental_Education_Level	90
Distance_from_Home	68
Gender	1
Exam_Score	1

dtype: int64

```
In [ ]: df = df.fillna(0)
df
```

```
Out[ ]:
```

	Hours_Studied	Attendance	Parental_Involvement	Access_to_Resources	I
0	23	84	Low	High	
1	19	64	Low	Medium	
2	24	98	Medium	Medium	
3	29	89	Low	Medium	
4	19	92	Medium	Medium	
...	...	...	...	...	...
6602	25	69	High	Medium	
6603	23	76	High	Medium	
6604	20	90	Medium	Low	
6605	10	86	High	High	
6606	15	67	Medium	Low	

6607 rows × 20 columns

```
In [ ]: df.columns = df.columns.str.lower()
df.head()
```

```
Out[ ]:
```

	hours_studied	attendance	parental_involvement	access_to_resources	extra
0	23	84	Low	High	
1	19	64	Low	Medium	
2	24	98	Medium	Medium	
3	29	89	Low	Medium	
4	19	92	Medium	Medium	

```
In [ ]: df.shape
```

```
Out[ ]: (6607, 20)
```

```
In [ ]: print("Before:", df.shape)
df = df.drop_duplicates()
print("After:", df.shape)
```

Before: (6607, 20)

After: (6605, 20)

```
In [ ]: df.columns = df.columns.str.lower()
df.loc[:, "sleep_hours"] = df["sleep_hours"].fillna(df["sleep_hours"].mean())
df["sleep_hours"]
```

Out[]:

sleep_hours	
0	7.0
1	8.0
2	7.0
3	8.0
4	6.0
...	...
6602	7.0
6603	8.0
6604	6.0
6605	6.0
6606	9.0

6605 rows × 1 columns

dtype: float64

```
In [ ]: df.loc[:, "physical_activity"] = df["physical_activity"].fillna(df["physical_activity"])
```

Out[]: **physical_activity**

0	3.0
1	4.0
2	4.0
3	4.0
4	4.0
...	...
6602	2.0
6603	2.0
6604	2.0
6605	3.0
6606	4.0

6605 rows × 1 columns

dtype: float64

In []: df.dtypes

Out[]:

0

hours_studied	int64
attendance	int64
parental_involvement	object
access_to_resources	object
extracurricular_activities	object
sleep_hours	float64
previous_scores	int64
motivation_level	object
internet_access	object
tutoring_sessions	float64
family_income	object
teacher_quality	object
school_type	object
peer_influence	object
physical_activity	float64
learning_disabilities	object
parental_education_level	object
distance_from_home	object
gender	object
exam_score	float64

dtype: object

```
In [ ]: df.loc[:, "previous_scores"] = df["previous_scores"].astype(float)
df["previous_scores"]
```

/tmp/ipykernel_9327/1486045474.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
df["previous_scores"] = df["previous_scores"].astype(float)

Out[]: **previous_scores**

0	73.0
1	59.0
2	91.0
3	98.0
4	65.0
...	...
6602	76.0
6603	81.0
6604	65.0
6605	91.0
6606	94.0

6605 rows × 1 columns

dtype: float64

```
In [ ]: df.loc[:, "gender"] = df["gender"].astype(str)
df["gender"]
```

Out[]: **gender**

0	Male
1	0
2	Male
3	Male
4	Female
...	...
6602	Female
6603	Female
6604	Female
6605	Female
6606	Male

6605 rows × 1 columns

dtype: object

In []: `df[df["attendance"] > 75]`

Out[]: **hours_studied attendance parental_involvement access_to_resources e**

0	23	84	Low	High
2	24	98	Medium	Medium
3	29	89	Low	Medium
4	19	92	Medium	Medium
5	29	84	Medium	Low
...	...	...	...	...
6600	12	98	Medium	Low
6601	20	83	Medium	Low
6603	23	76	High	Medium
6604	20	90	Medium	Low
6605	10	86	High	High

4060 rows × 20 columns

In []: `df[(df["attendance"] > 75) & (df["previous_scores"] >= 75)]`

	hours_studied	attendance	parental_involvement	access_to_resources	enrollment
2	24	98	Medium	Medium	
3	29	89	Low	Medium	
8	17	94	Medium	High	
10	17	80	Low	High	
11	21	83	Medium	Medium	
...	...	...	...	...	...
6592	29	100	Medium	Low	
6594	9	90	High	High	
6595	28	78	Medium	High	
6603	23	76	High	Medium	
6605	10	86	High	High	

2050 rows × 20 columns

```
In [ ]: from google.colab import files
df.to_csv('cleaned_studentperfor2.csv', index=False)
files.download('cleaned_studentperfor2.csv')
```

Conclusion:- This practical provided a clear understanding of data ingestion and cleaning using the Pandas library. It demonstrated how to load datasets, identify and handle missing values, remove duplicates, and transform data by renaming columns and changing data types. Additionally, filtering data and saving the cleaned dataset helped in preparing data for further analysis. Overall, the practical improved the ability to organize and clean raw data, which is an essential step in any data analysis or machine learning process.